

Person 5 -- Frontier Layer

NHAA Trauma Triage System -- Standalone Build Specification

How to Use This Document

This is a complete, standalone build specification. Build every phase directly, in order, without waiting for or requiring live coordination with anyone else.

Your Role and an Honest Constraint

You build four frontier capabilities: Cross-Call Pattern Linkage, the Live Interview Coach, the Outcome Feedback Loop, and the ASHA Physical Safety Bridge. The first three are fully buildable as real, complete, working software today. The fourth is different: you can and will build the complete, real software layer for it, but an actual live connection to India's real ASHA community-health-worker network requires a genuine government partnership that does not exist yet and cannot be created by writing code. Build the real interface as if that partnership already existed; state plainly in your final documentation that the live connection itself is the one piece requiring outside partnership, not engineering time.

Phase 1 -- Cross-Call Pattern Linkage: Design and Build

Define, in a code comment at the top of your file, exactly which features are safe to compare across calls (examples: "named individual mentioned" as a normalized string, "locality type" as a category like village/town/city, "intimidation method category" as one of a fixed short list) and note explicitly that raw transcript text, audio, or anything that could identify a specific caller is never included in this comparison.

```
Build a function check_pattern_correlation(new_case: dict,
recent_cases: list) -> dict where each case is a dict of only the safe
categorical features defined above. Compute a simple similarity score
(count of matching categorical fields divided by total fields compared)
between new_case and each entry in recent_cases. If any comparison
scores above 0.6, return {"flagged": true, "matched_case_ids":
[...]}; otherwise {"flagged": false}.
```

Test: build three synthetic case dicts that share the same "named individual" and "locality type" fields, confirm they're flagged against each other; build five unrelated synthetic cases with no shared fields, confirm none are flagged.

Phase 2 -- Live Interview Coach

Build a function `generate_coaching_cue(risk_vector: dict, recent_silence_event: dict, recent_question_type: str) -> str` that returns one of these fixed cue strings based on simple rule conditions you define: if `recent_silence_event` shows `placement == "mid_sentence"` and `duration_sec > 2`, return "Let this silence continue -- do not fill it." If `risk_vector["fear_of_retaliation"] > 0.5` or `risk_vector["intimidation"] > 0.5` and `recent_question_type == "yes_no"`, return "Consider an open-ended question here." Otherwise return an empty string (no cue). Build a small test script that feeds it three different fake input combinations and confirms the correct cue (or no cue) comes back each time.

Phase 3 -- Outcome Feedback Loop

Build a simple SQLite table `outcome_followups` with columns `case_id`, `scheduled_date`, `consent_given`, `outcome_response`, `reviewed_by_human`. Write a function `schedule_followup(case_id, weeks_ahead=3)` that inserts a row with a future date. Write a function `record_outcome(case_id, response_text)` that updates the row with the caller's response, leaving `reviewed_by_human` as `False`. Write a function `get_pending_human_reviews()` that returns every row where `outcome_response` is filled in but `reviewed_by_human` is still `False` -- this is the queue a real mental-health professional would work through; do not build anything that automatically changes any scoring weights based on this data. Test: schedule a followup, record a fake outcome, confirm it appears in the pending-review queue.

Phase 4 -- The ASHA Bridge Software Layer

Build a function `prepare_asha_handoff(case_id: str, action_state: dict, case_summary: dict) -> dict` that: first checks `action_state["state"] == "executed"` and `action_state["requires_senior"] == True` (confirming this case genuinely passed the same senior-approval gate as police/witness-protection actions -- if not, raise an error and refuse to proceed), then returns a structured handoff object: `{"case_id": ..., "priority": "critical", "briefing": case_summary, "requested_action": "discreet welfare check", "framing_note": "present as routine health visit"}`.

Build a second function `mock_asha_receiving_system(handoff: dict) -> None` that simply logs the received handoff to a text file, standing in for a real receiving system. Test the full flow: build a fake `action_state` and `case_summary`, call `prepare_asha_handoff`, confirm it correctly refuses if the action wasn't senior-approved, confirm it correctly produces the handoff object if it was, and confirm `mock_asha_receiving_system` logs it correctly.

Phase 5 -- Package for Merge and Write the Honesty Note

Combine all four capabilities into one module `nhaa_frontier.py`. Write a `README_frontier.md` documenting each function exactly as you did the code, and include this exact paragraph, unedited, in your README's final section: "The Cross-Call Pattern Linkage, Live Interview Coach, and Outcome Feedback Loop modules in this package are complete, real, working software. The ASHA Physical Safety Bridge module is also complete, real, working software for everything up to and including the handoff object -- what it does not include, because it cannot be built in code, is an actual live connection to a real ASHA worker network, which requires a state-level institutional partnership outside this project's scope. This module is fully ready to connect to such a partnership the moment one exists."